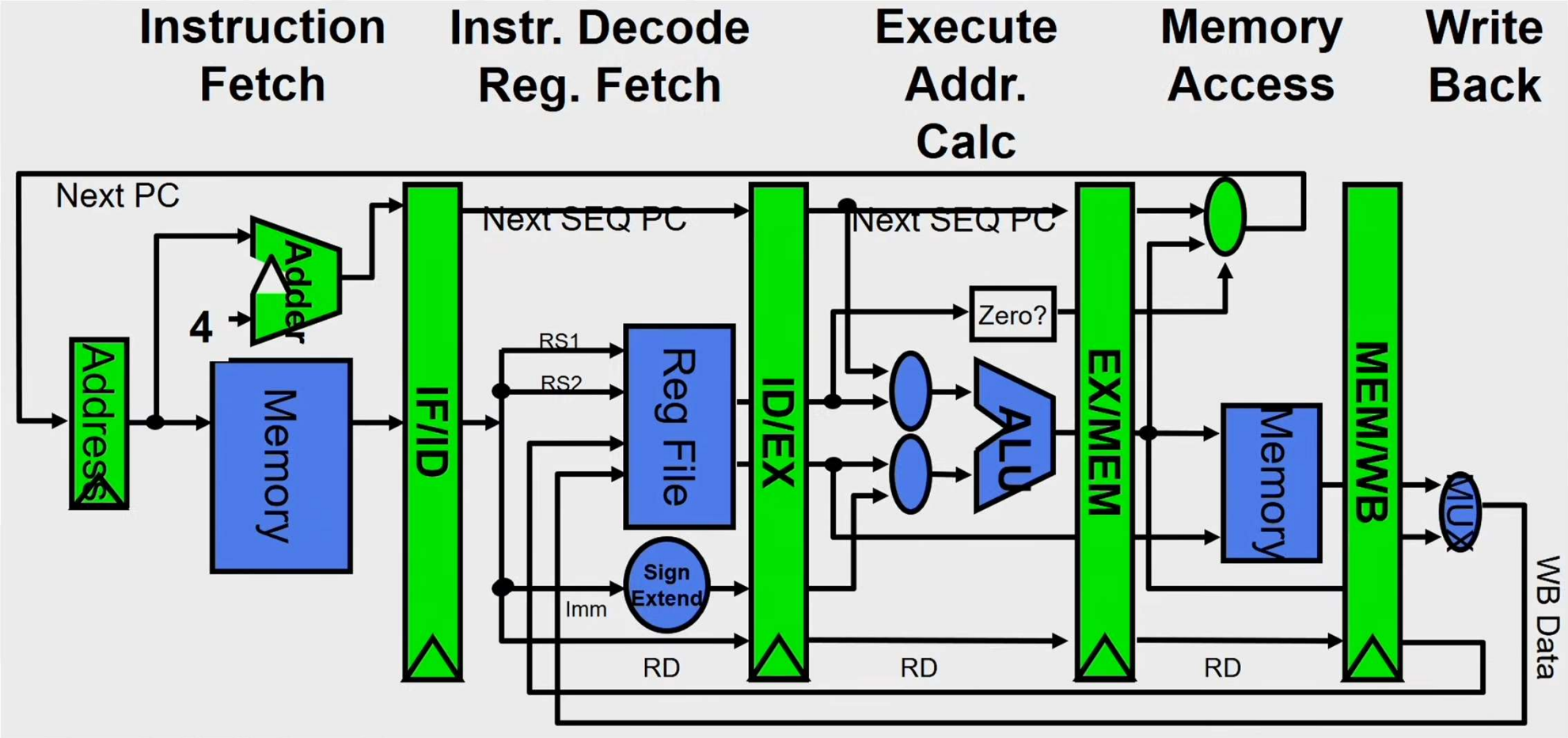
System and Devices (SYS 3)

Lecture 6: Pipeline Hazards

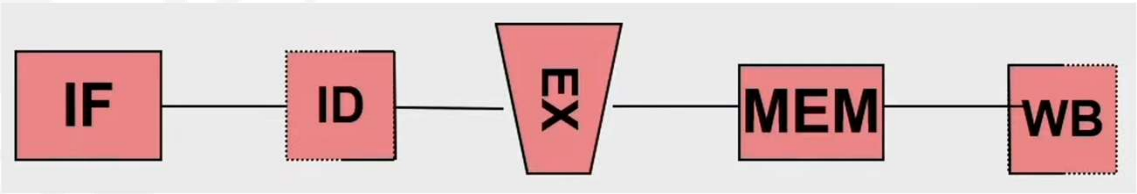
Dr Pengcheng Liu

Pipelined RISC Data Path



5 Steps of RISC Data Path

- Each instruction can take at most 5 lock cycles
- Instruction fetch cycle (IF)
- Instruction decode/register fetch cycle (ID)
- Execution/Effective address cycle (EX)
- Memory access cycle (MEM)
- Write-back cycle (WB)



Visualizing Pipelining

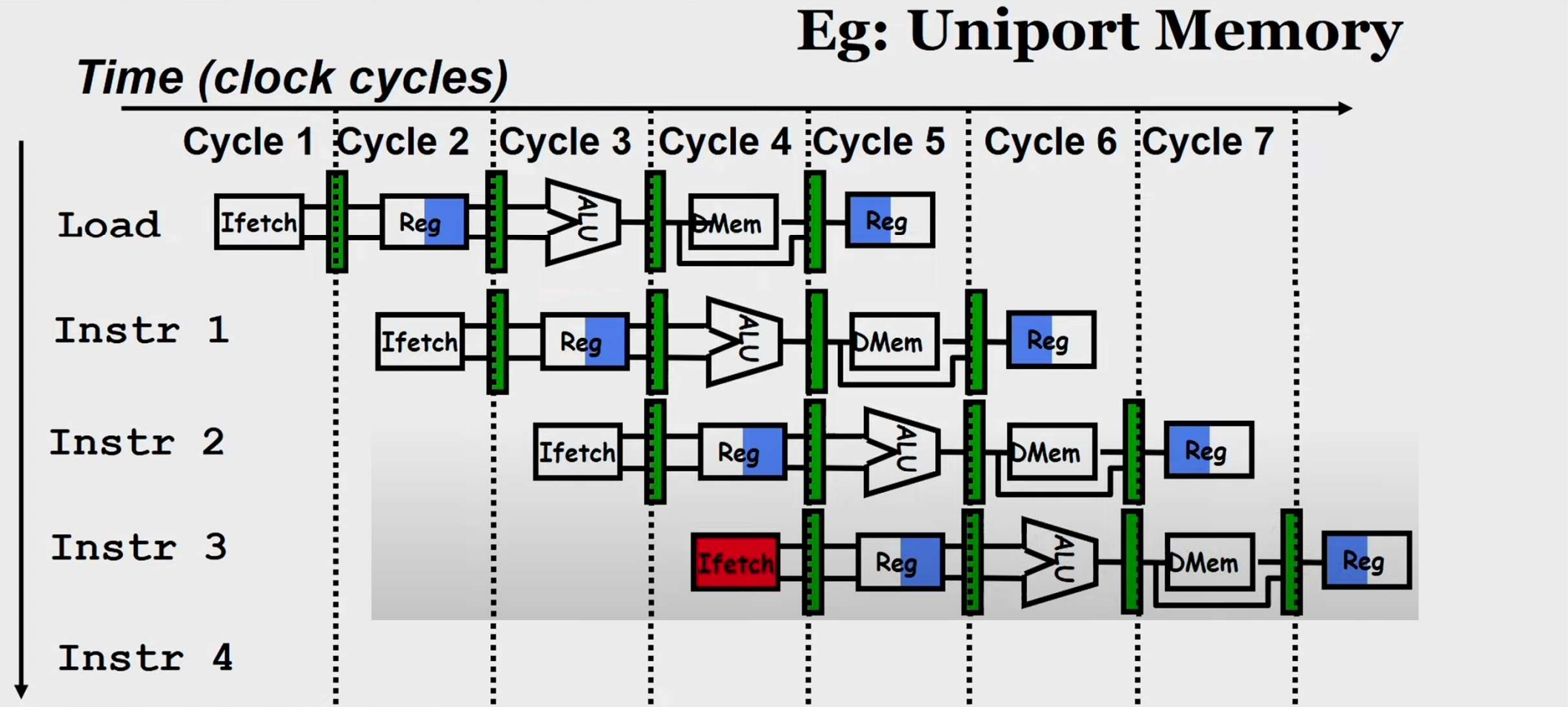
	Clock number							
Instruction number	1	2	3	4	5	6	7	8
i	IF	ID	EX	MEM	WB			
i+1		IF	ID	EX	MEM	WB		
i+2			IF	ID	EX	MEM	WB	
i+3				IF	ID	EX	MEM	WB
i+4					IF	ID	EX	MEM

Limits to Pipelining

Hazards: circumstances that would cause incorrect execution if next instruction if fetched and executed

- **Structural hazards**: Different instructions, at different stages, in the pipeline want to use the same hardware resource
- **Data hazards**: An instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline
- **Control hazards**: Succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline

Structural Hazard



Structural Hazard if one read port in memory

Resolving Structural Hazards

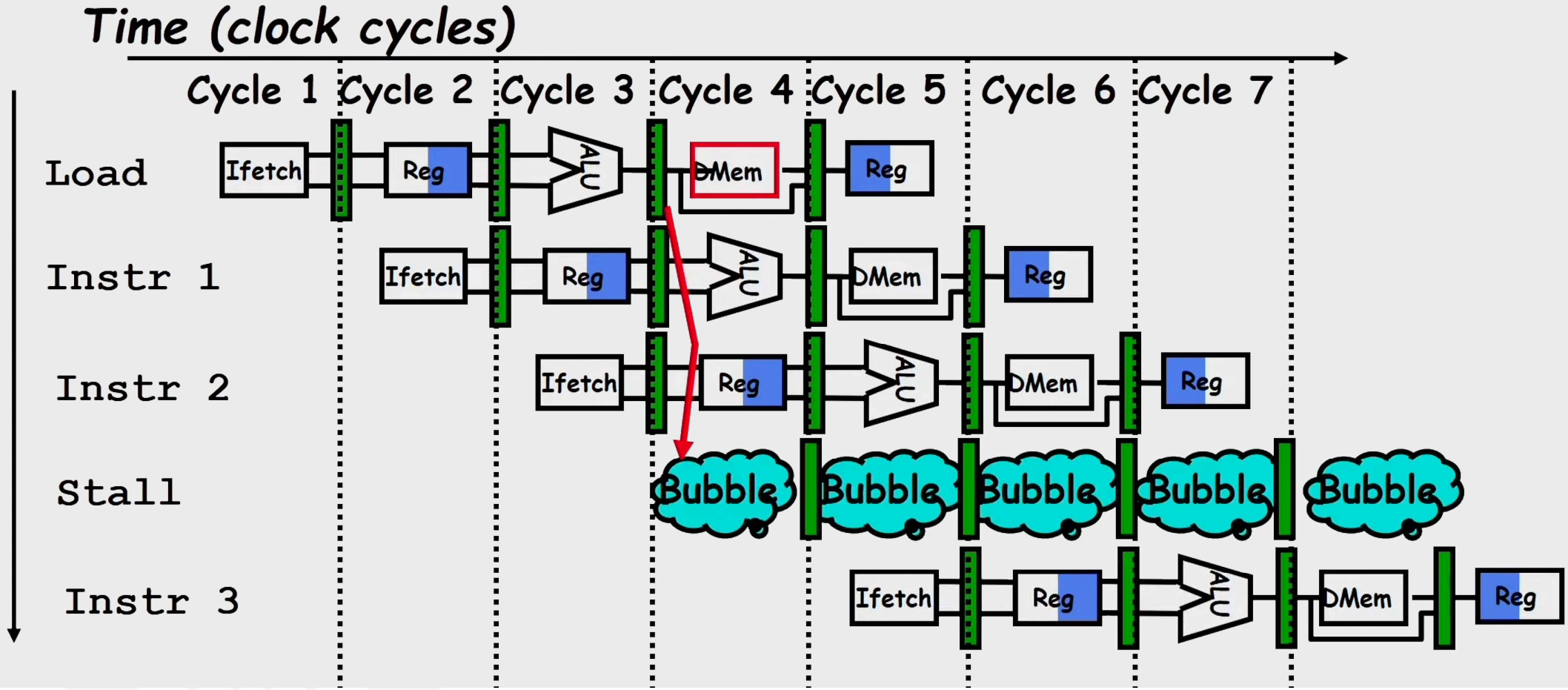
- Eliminate the use same hardware for two different thins at the same time
- **Solution 1: Wait**

  - must detect the hazard

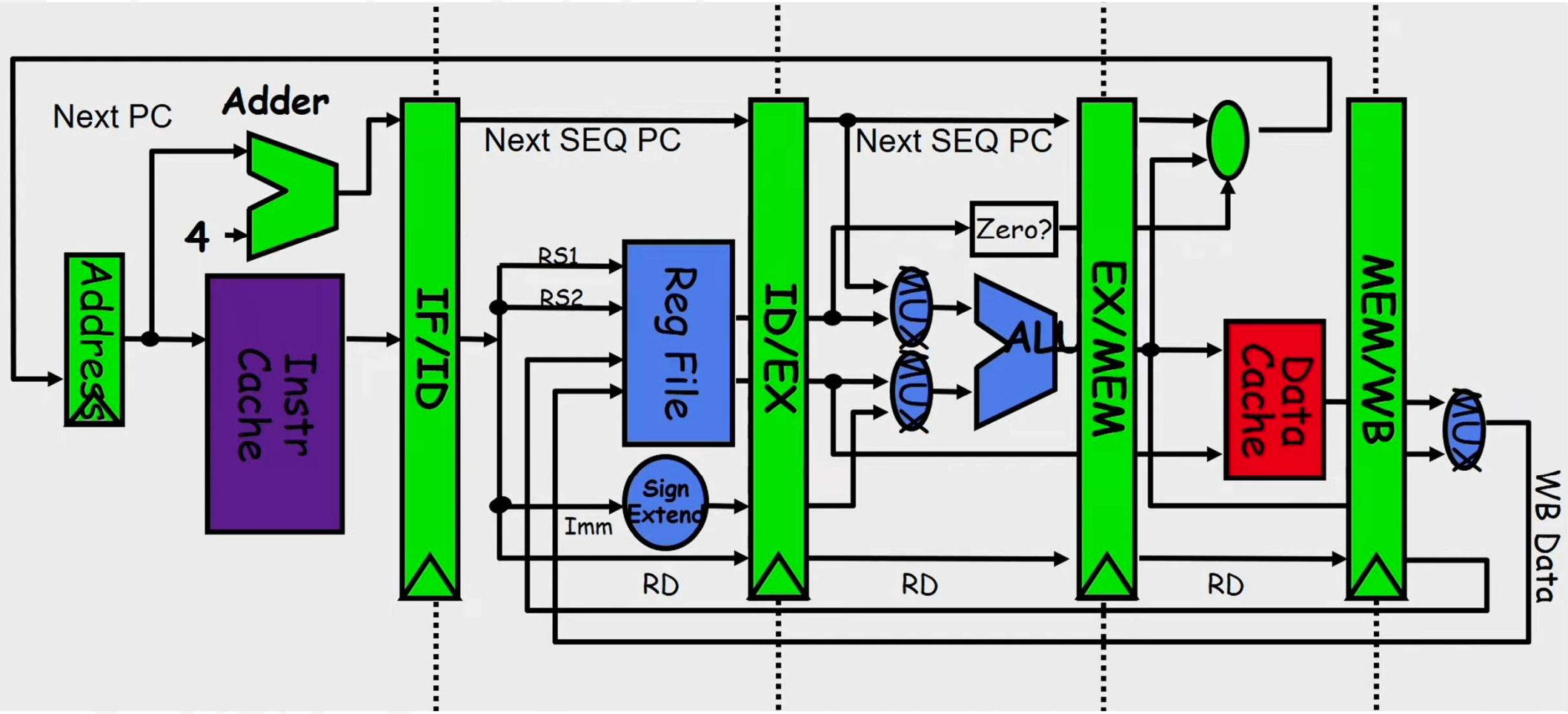
  - must have mechanism to stall
- **Solution 2: Duplicate hardware**

  - Multiple such units will help both instruction to progress

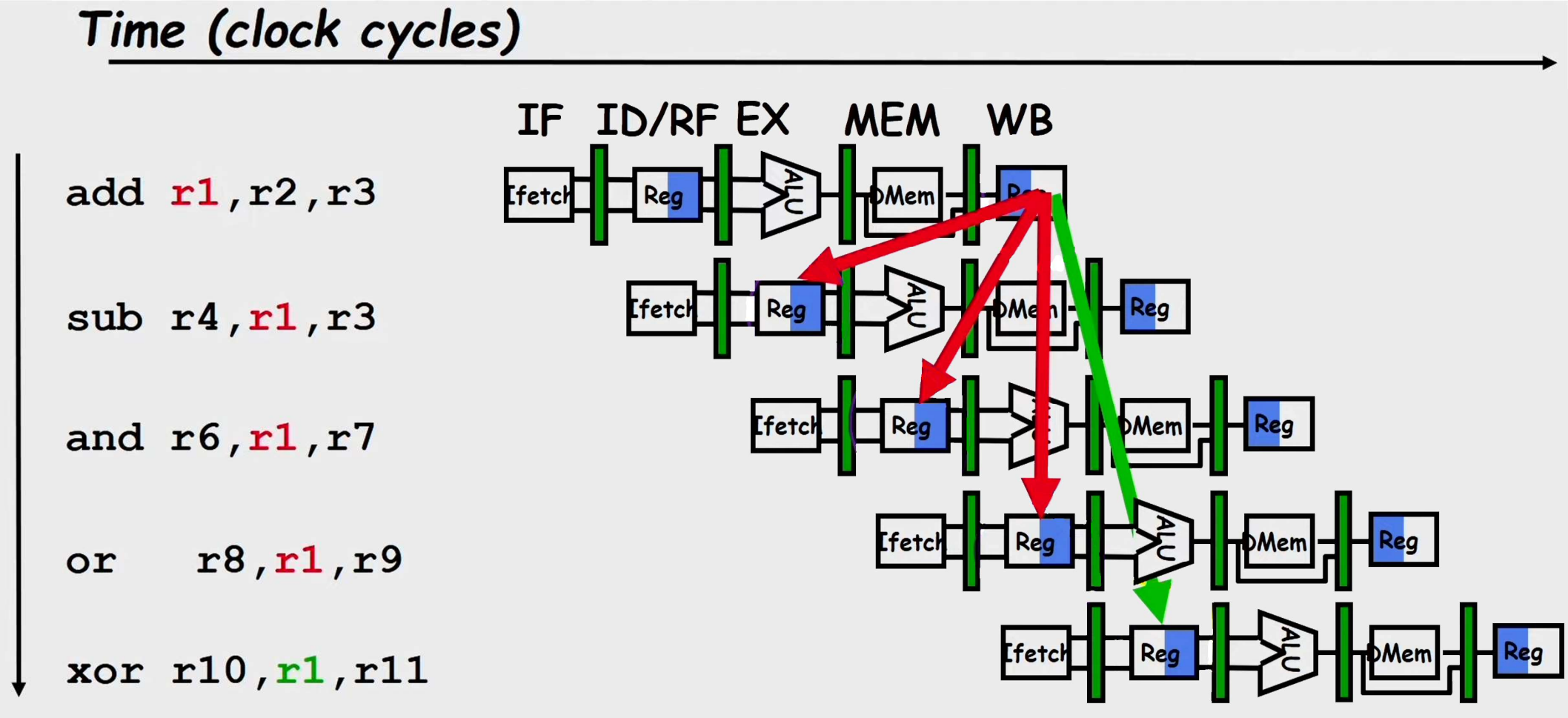
Detecting & Resolving Structural Hazards



Eliminating Structural Hazards at Design



Data Hazard


Time (clock cycles)
IF
ID/RF
EX
MEM
WB
add r1,r2,r3
sub r4,r1,r3
and r6,r1,r7
or r8,r1,r9
xor r10,r1,r11
Reg
ALU
DMem

Three Generic Data Hazards

- Read After Write (RAW)

Instruction J tries to read operand before Instruction I writes it

```
I : add r1,r2,r3
J: sub r4,r1,r3
```

- Caused by a data dependence
- This hazard results from an actual need for communication

Three Generic Data Hazards

- Write After Read (WAR)

Instruction J writes operand before Instruction I reads it

```
I: sub r4,r1,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

- Called an anti-dependence by compiler writers
- This results from reuse of the name r1
- Can't happen in 5 stage pipeline because:

- All instructions take 5 stages, and
- Reads are always in stage 2, and
- Writes are always in stage 5

Three Generic Data Hazards

- Write After Write (WAW)

Instruction J writes operand before Instruction I writes it

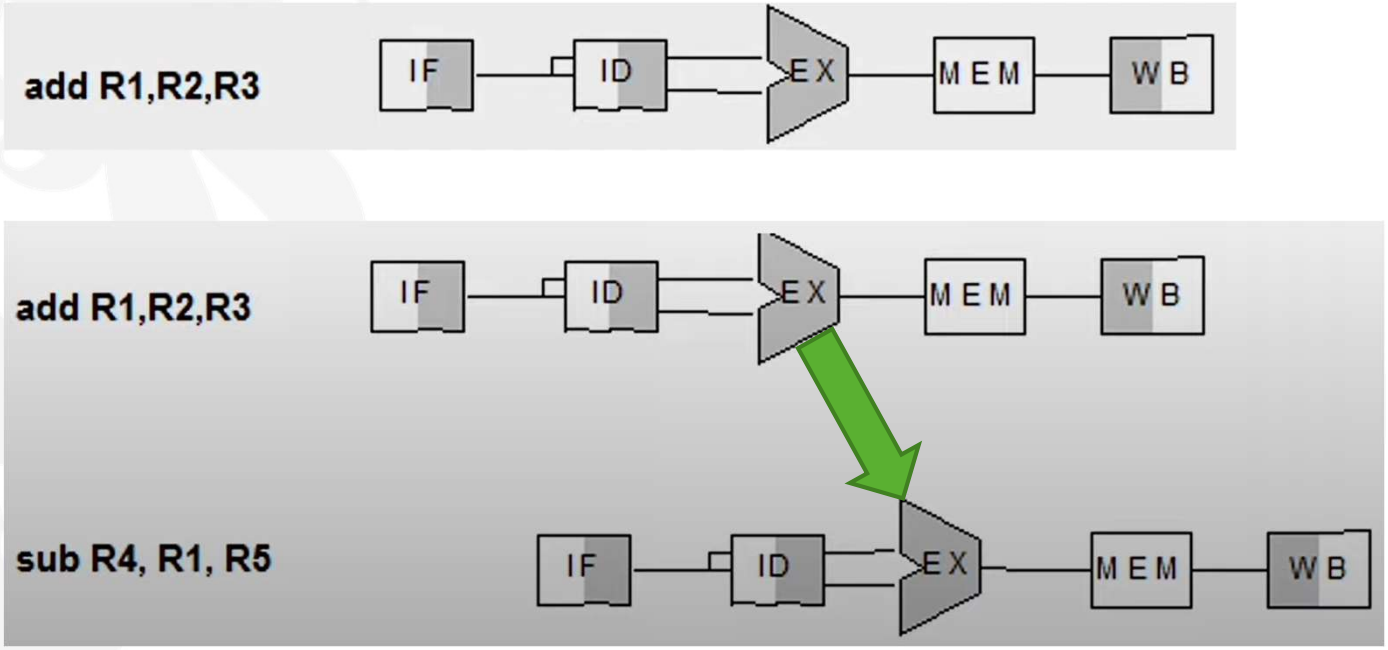
```
I: sub r1,r4,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

- Called an output dependence
- This also results from reuse of the name r1
- Can't happen in 5 stage pipeline because:
  - All instructions take 5 stages, and
  - Writes are always in stage 5

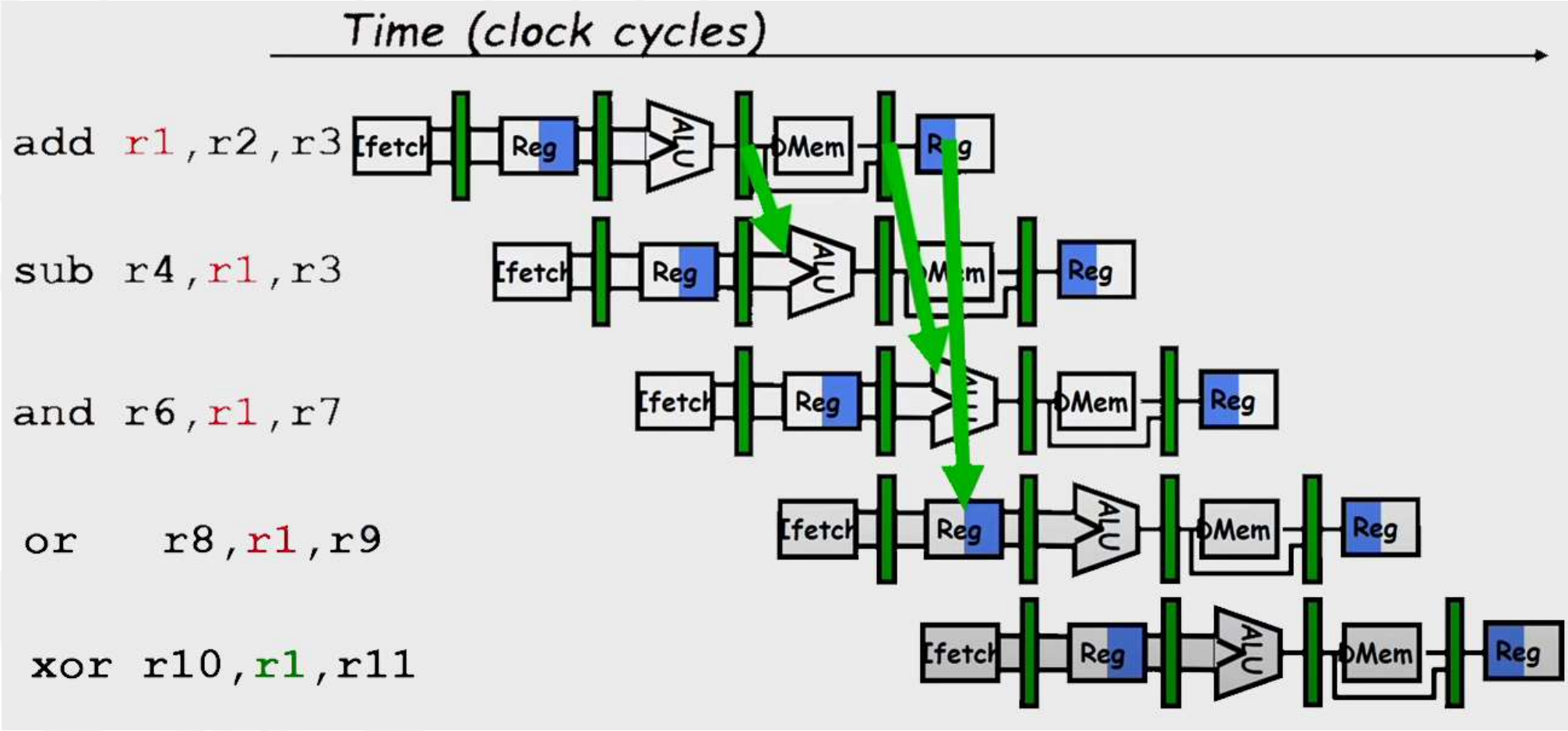
WAR and WAW happens in out of order pipes

How to Handel Data Hazard?

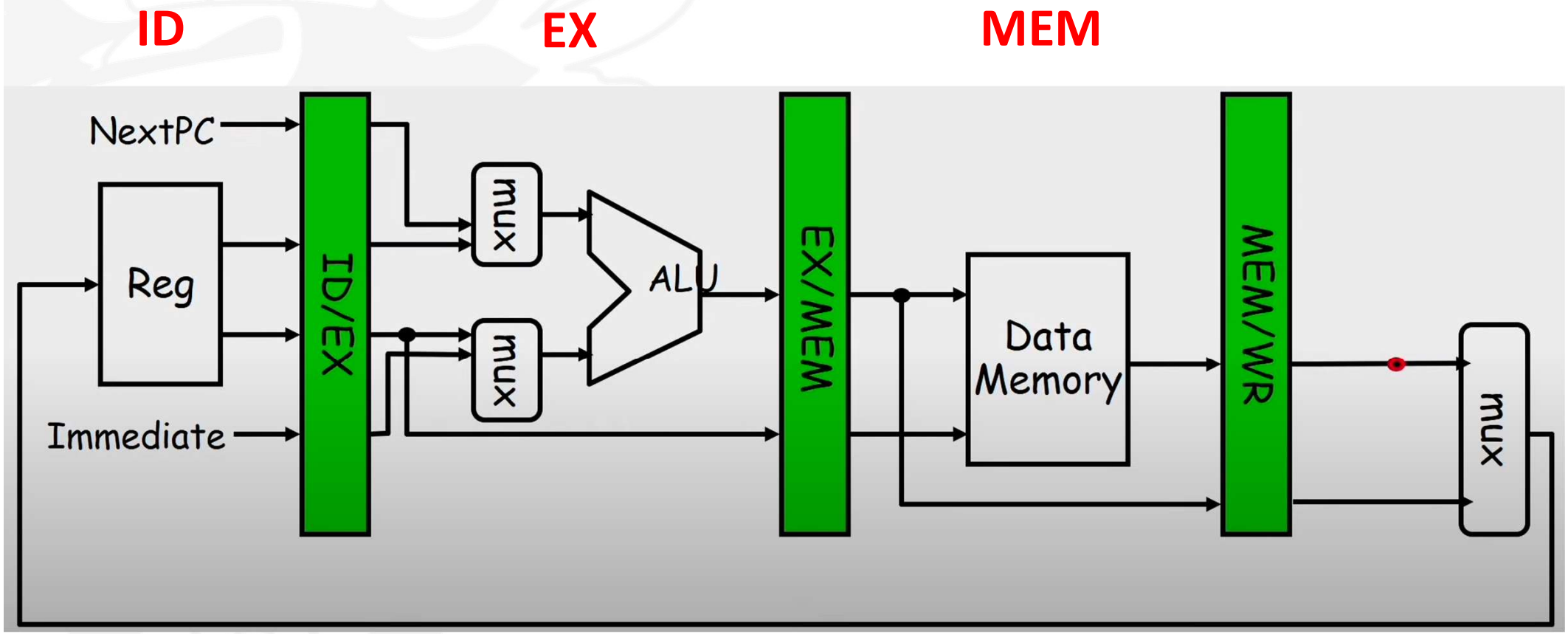
- Data hazard: instruction needs data from the result of a previous instruction still executing in pipeline
- Solution: Forward data if possible



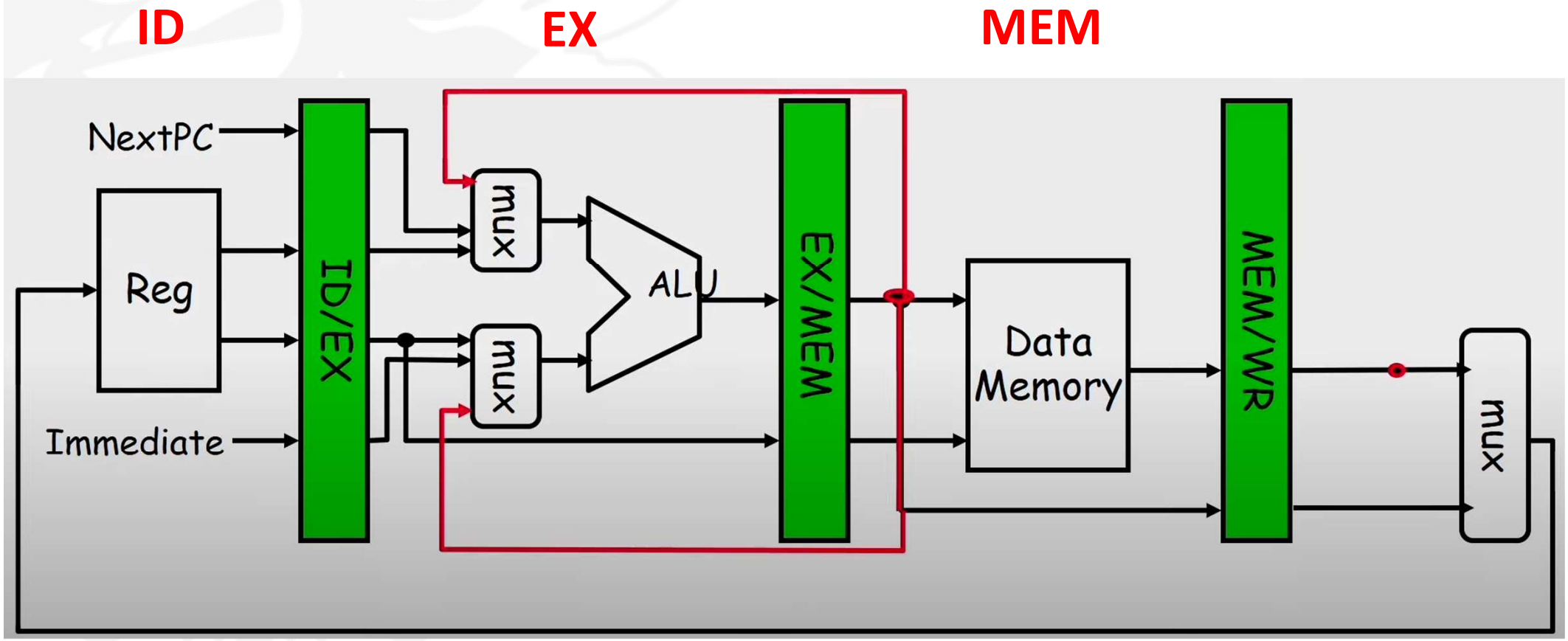
Operand Forwarding to Avoid Data Hazard



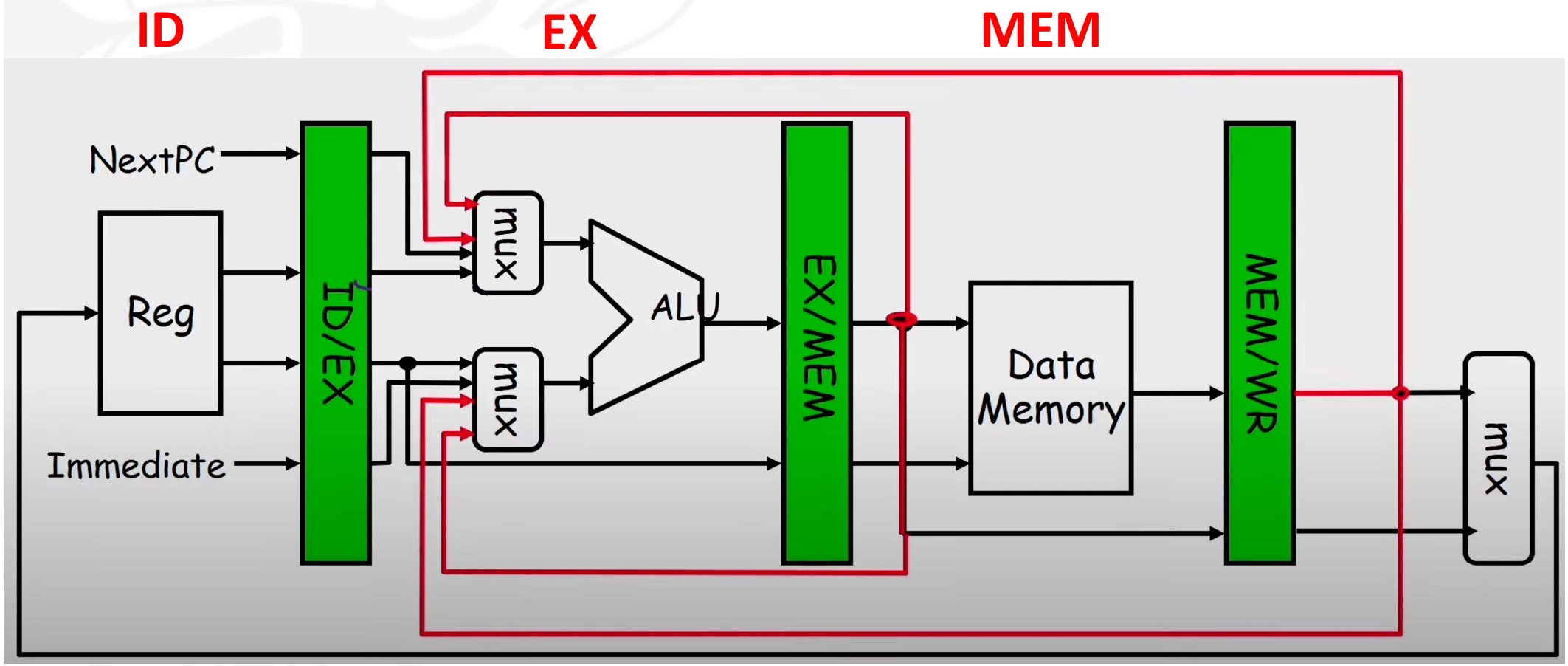
Hardware Change for Forwarding



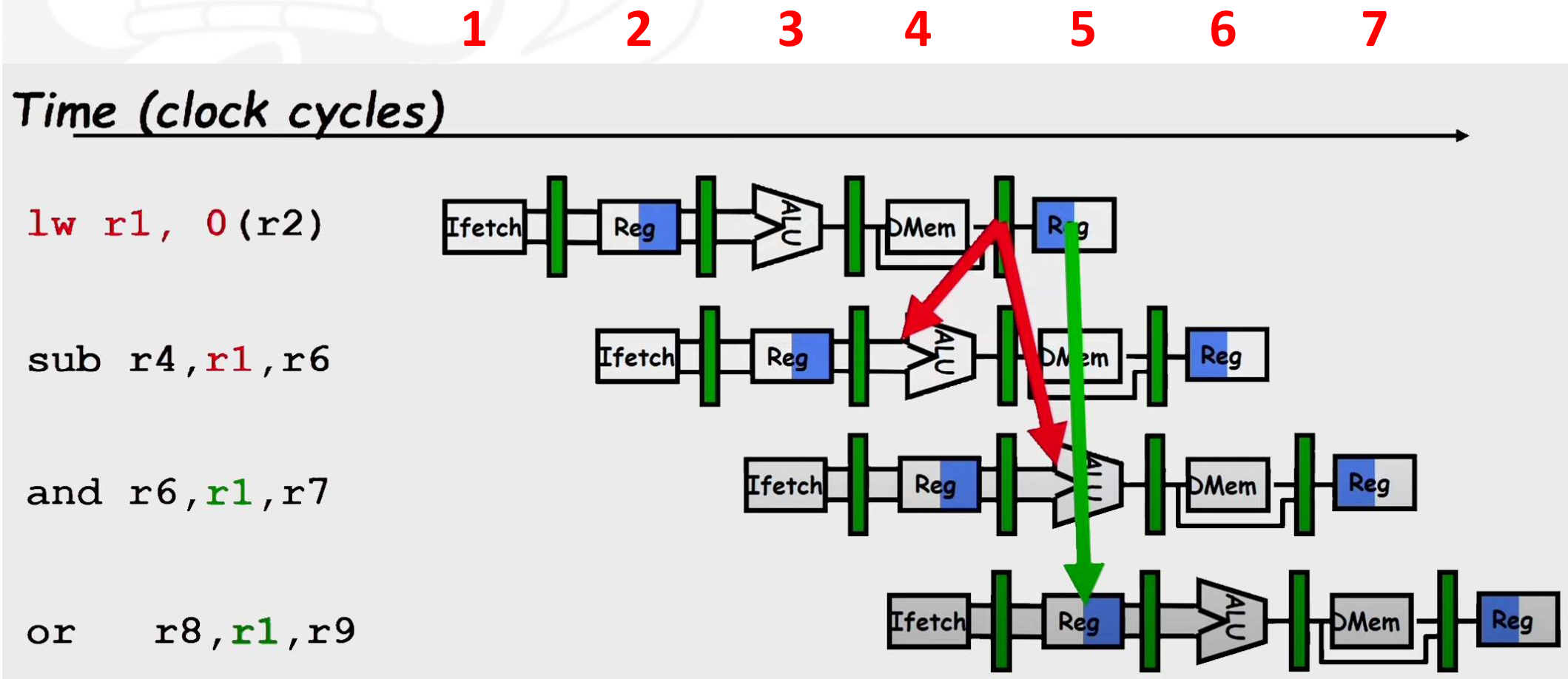
Hardware Change for Forwarding



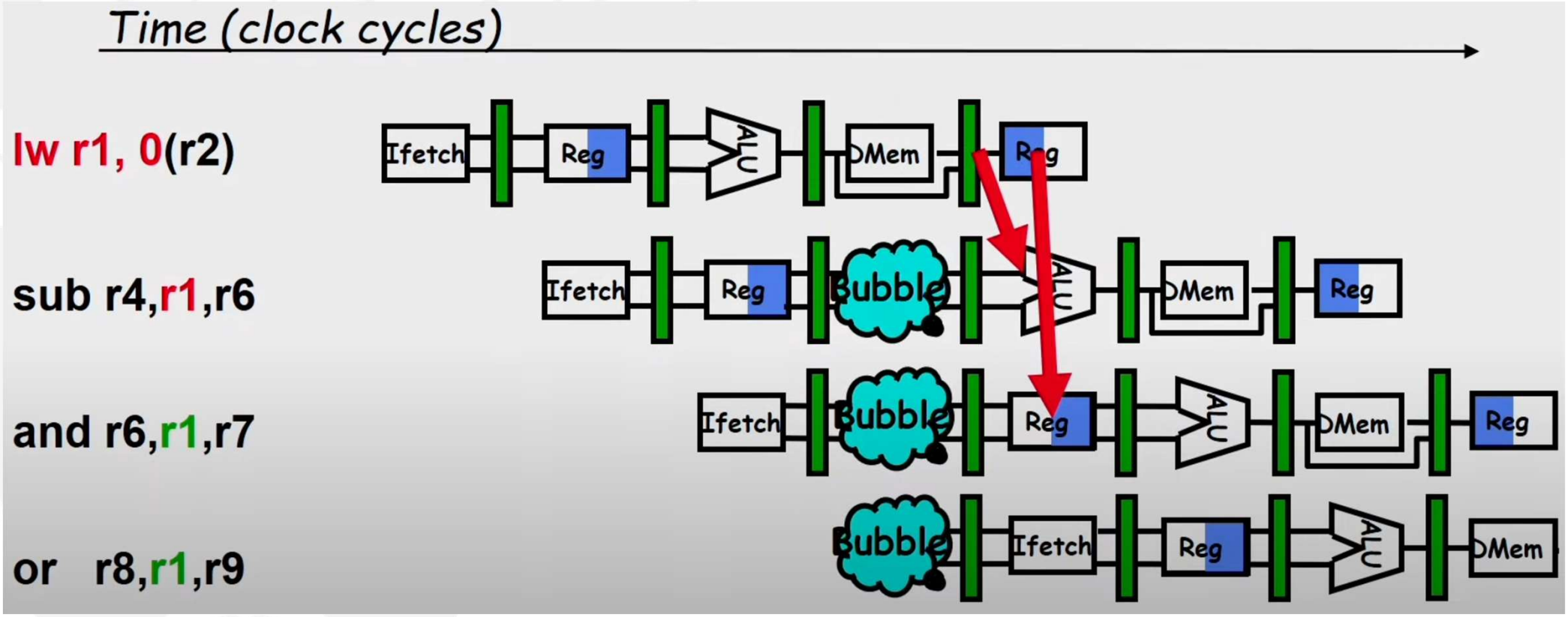
Hardware Change for Forwarding



Data Hazard even with Operand Forwarding



Resolving the Load-ALU Hazard



Software Scheduling for Load Hazards

Assume a, b, c, d, e and f in memory.

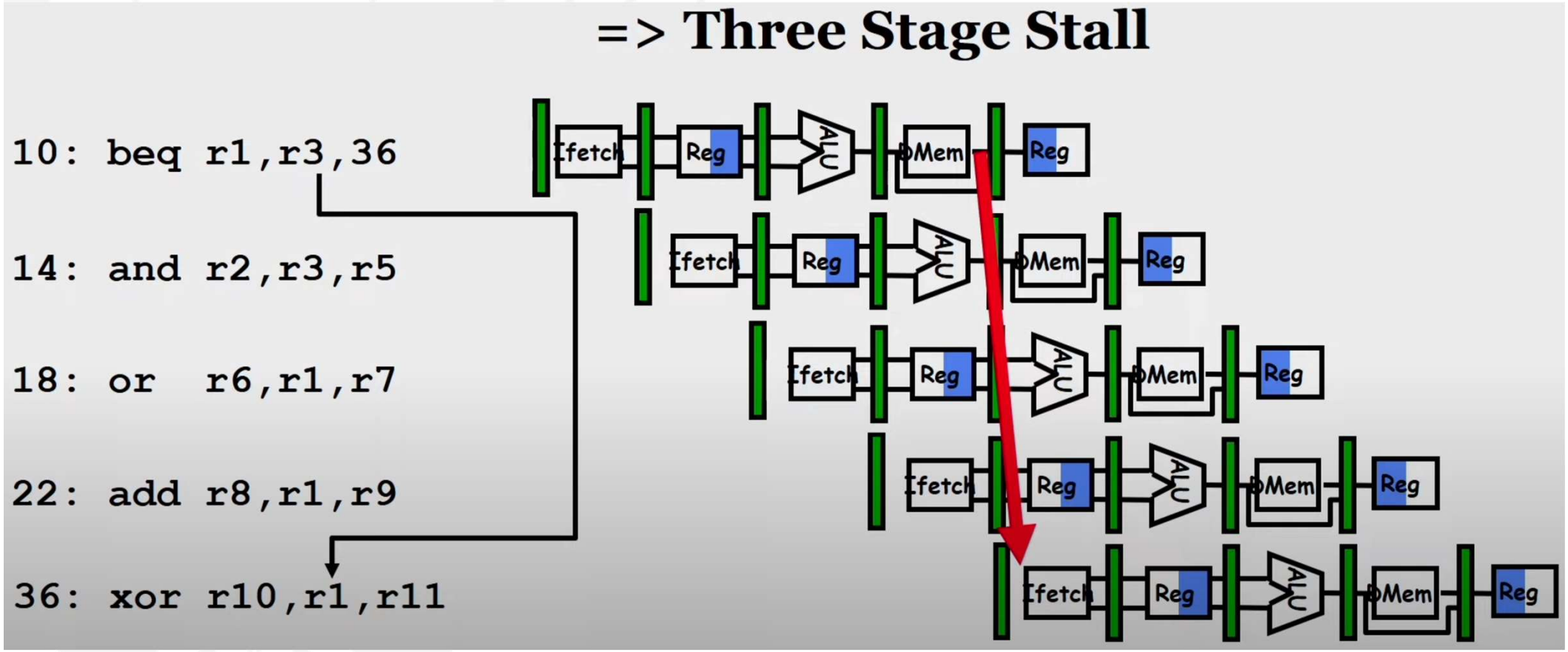
a = b + c;

d = e – f;

5		LW Rb,b
6	Stall	LW Rc,c
8		ADD Ra,Rb,Rc
9		SW a,Ra
10		LW Re,e
11	Stall	LW Rf,f
13		SUB Rd,Re,Rf
14		SW d,Rd

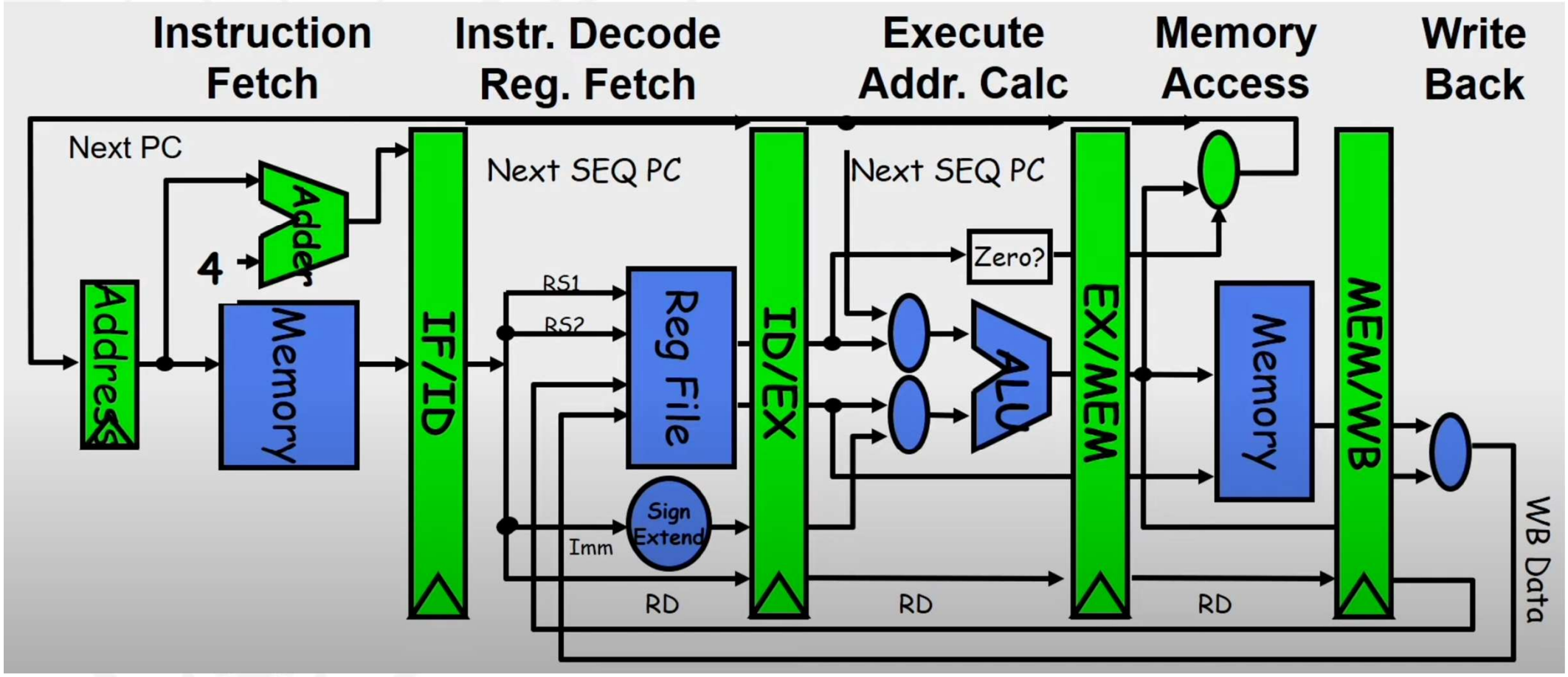
LW Rb,b	5
LW Rc,c	6
LW Re,e	7
ADD Ra,Rb,Rc	8
LW Rf,f	9
SW a,Ra	10
SUB Rd,Re,Rf	11
SW d,Rd	12

Control Hazard on Branches



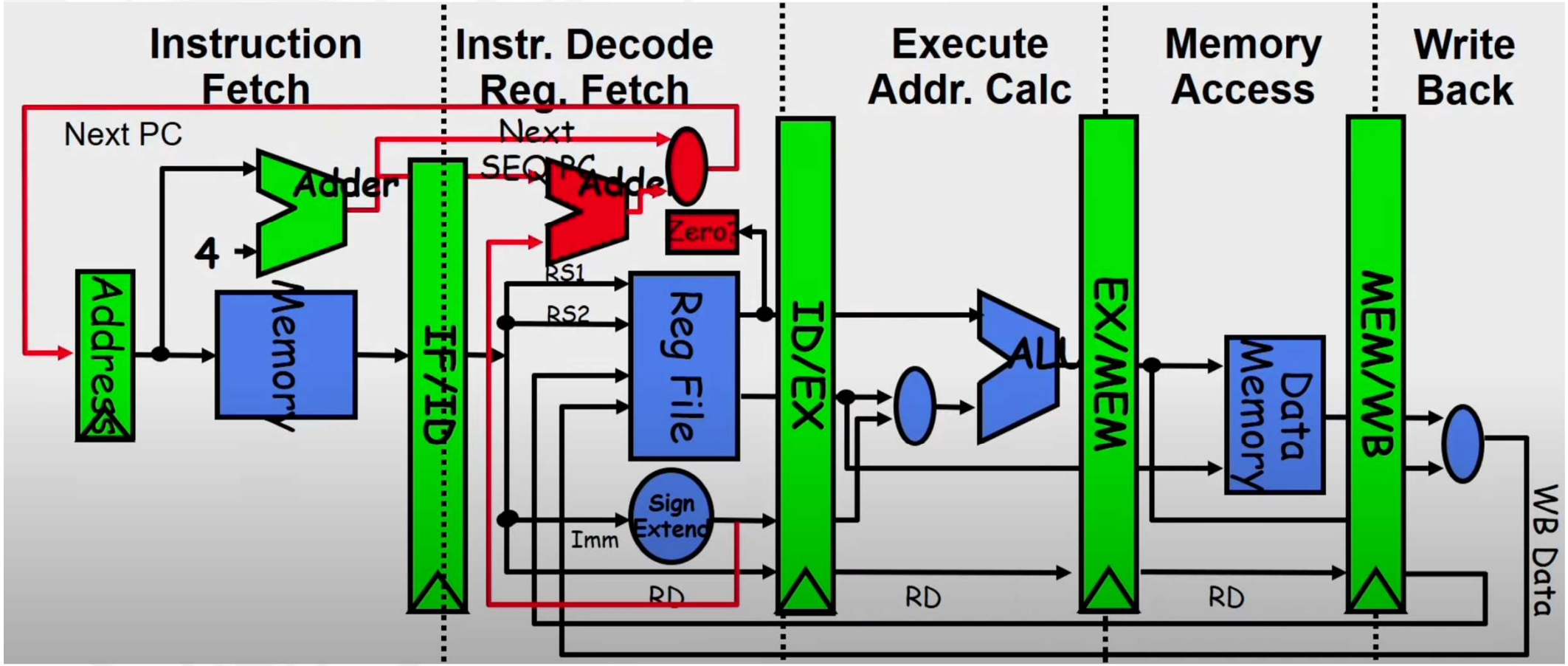
Conventional 5-Stage Pipeline

Branching at 4th Stage



Branch Optimized 5-Stage Pipeline

Branching at 2nd Stage



Summary

Suggested Readings

Computer Architecture: A Quantitative Approach (5th Edition)
by John Hennessy and David A Patterson:

- pp. C-11 – C-21: Structural Hazards & Data Hazards